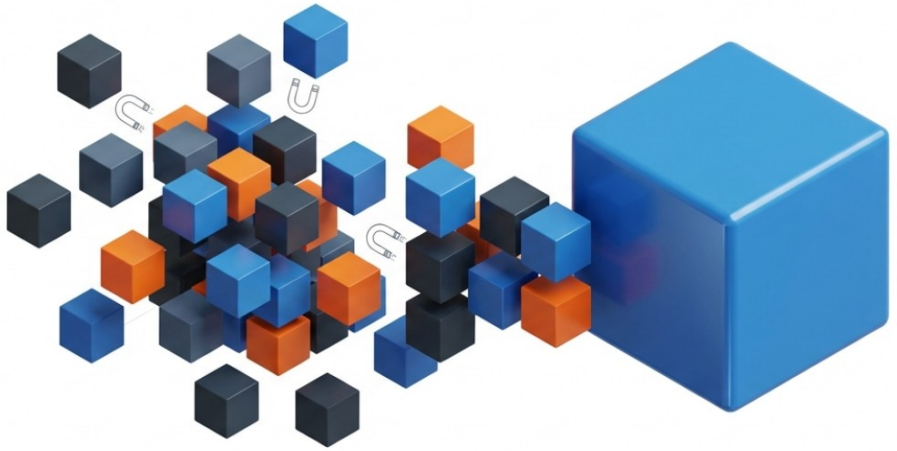
CHAPTER 2

# BASIC ELEMENTS IN C++

C++ Programming • 261 • 2026-2027

Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering



For academic use only

# Learning Objectives

By the end of this lesson, students will be able to...

1. **Understand the basic structure of a C++ program**

   Master the basic structure, role of the main() function, preprocessor directives, and use of the cout object.

2. **Distinguish basic data types and operators**

   Understand basic data types, principles of IEEE 754 floating-point representation, and operator precedence.

3. **Declare variables, assign values, and handle input**

   Correctly use variable declaration syntax, assignment statements, increment/decrement operators, and the cin object for data input.

4. **Use mathematical library functions and format output**

   Apply functions in the <cmath> library and stream manipulators (setw, setprecision) to format output display.

# Lesson Roadmap

1

## Basic Structure

C++ Program & main() function

2

## Types & Operators

Data Types & IEEE 754 Standard

3

## Variables & Input

Variable Declarations & cin Statement

4

## Libraries & Formatting

Math Functions & cout Formatting

01

# INTRODUCTION TO A SIMPLE C++ PROGRAM

**Get familiar with the fundamental structure of a C++ program, from the main() function and standard I/O libraries to printing your first message on screen.**

# A simple program

Header file

```
#include <iostream>
using namespace std;
int main()
{
      cout << "Hello world!";
      return 0;
}
```

- A header file is a file with an extension of .h that is included as part of a program. It notifies the compiler that a program uses run-time libraries.
- All statements in C++ must end with a semicolon.
- The `iostream` classes are used for giving C++ programs input and output capabilities.
- The header file for the `iostream` class is `iostream.h`.

# The main() function

## Definition / Principles

- The `main()` function is a special function that runs automatically when a program first executes.
- All C++ programs must include one `main()` function. All other functions in a C++ program are executed from the `main()`.
- The first line of the function, in this case int `main()` is called a function header line.
- The function header line contains three pieces of information: What type of data, if any, is returned from the function; The name of the function; What type of data, if any, is sent into the function.

## Examples

- `int main()`
- `int getNumber(void)`
- `int totalScore(int score1, int score2)`

# The main() function (cond.)

```
int main()
{
    <program   statements   in
   here>
      return 0;
}
```

- The line `return 0;` is included at the end of every main function.
- C++ keyword return is one of several means we will use to exit a function.
- When the return statement is used at the end of `main()`, the value 0 indicates that the program has terminated successfully.

# The cout Object

**The `cout` object is an output object that sends data given to it to the standard output display device.**

01 To send a message to the cout object, you use the following pattern:

```
cout << "text";
cout << "Hello " << "C++";
```

02 The insertion operator, <<, is used for sending text to an output device.

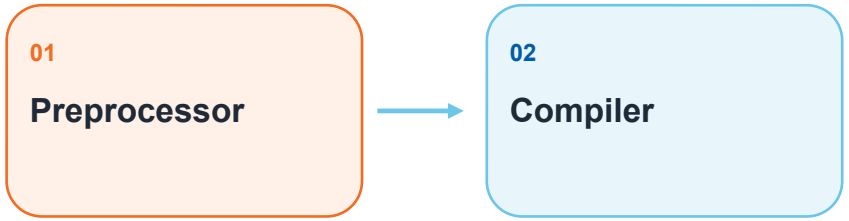
03 The text portion of a `cout` statement is called a text string.

# The iostream classes

```
#include <iostream>
using namespace
std;
```

- The `#include` statement is one of the several preprocessor directives that are used with C++.
- The statement `using namespace std;` tell the compiler where to find the header files.
- A `namespace` is a specific named section of code within a folder that is accessed by the compiler when it is looking for prewritten classes or function.

# Preprocessor directives




When the preprocessor encounters an `#include` statement, it places the entire contents of the designated file into the current file.
Preprocessor directives and include statements allow the current file to use any of the classes, functions, variables, and other code contained within the included file.

# I/O manipulator

```
cout << "Program type: console application" <<
endl;
cout << "Create with: Visual C++" << endl;
cout << "Programmer: Don Gesselin" << endl;
```

- An i/o manipulator is a special function that can be used with an i/o statement.
- The `endl` i/o manipulator is part of iostream classes and represents a new line character.

# Comments

## Line comments




- Created by adding two slashes (//) before the text you want to use as a comment.
- Example: `// this program calculates a square root`

## Block comments




- Span multiple lines. Such comments begin with `/*` and end with the symbols `*/`
- Example:

```
/* this program solves a quadratic
equation:
   ax2 + bx +c = 0  */
```

**Comments are lines that you place in your code to contain various types of remarks.**

02

# BASIC DATA TYPES AND OPERATORS

**Explore fundamental data types, arithmetic operators, and operator precedence rules in C++.**

# Data types

**A data type is the specific category of information that a variable contains.**

There are three basic data types used in C++:

01

### Integers

An integer is a positive or negative number with no decimal places. Examples: `-259 -13 0 200`

02

### Floating point numbers

03

### Characters

# Floating Point Numbers

## Definition

- A floating point number contains decimal places or is written using exponential notations.

## Examples

- `-6.16, -4.4, 2.7541, 10.5, 90.1e3, 0.89E8, -1.01e-1`
- Note: `90.1e3` means `90.1×103`

## Floating point number is always approximate value.

Nowadays, floating point numbers are handled using IEEE 754 standard, to represent and store. IEEE 754 standard divides the bits into three parts:

01 **A sign bit**

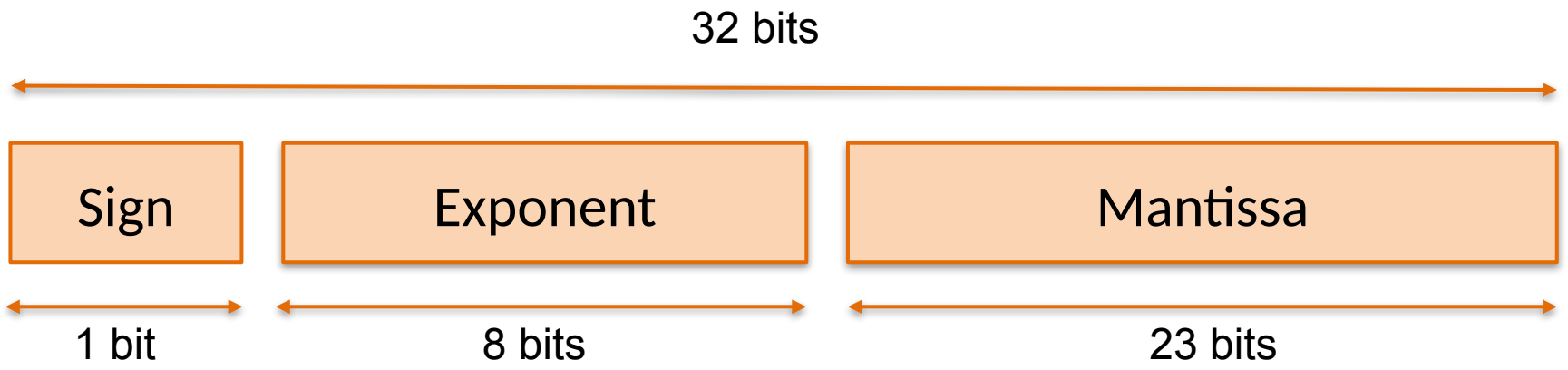
02 **An exponent**

Represents the magnitude (or scale) of the number

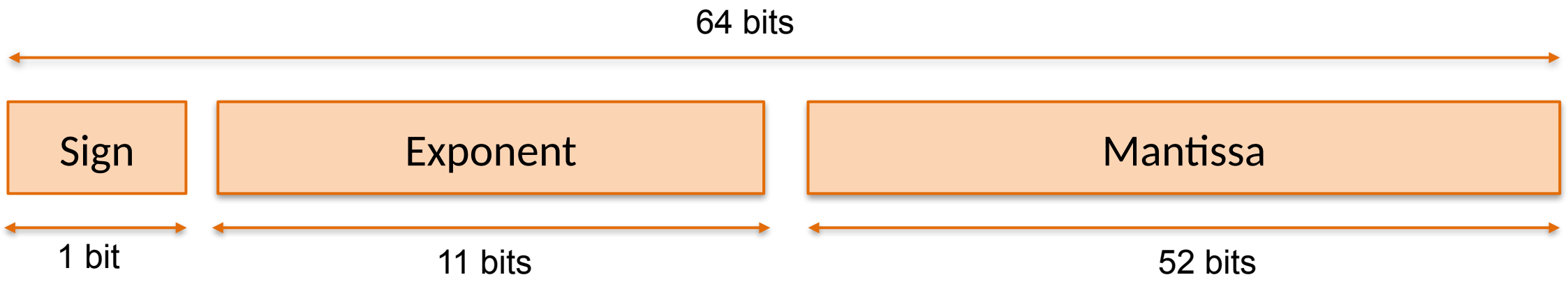
03 **A mantissa (or significand)**

Represents the fractional part of the number, with the leading "1" often implicit (not stored) to maximize precision

# Single-precision IEEE 754 Floating Point standard:

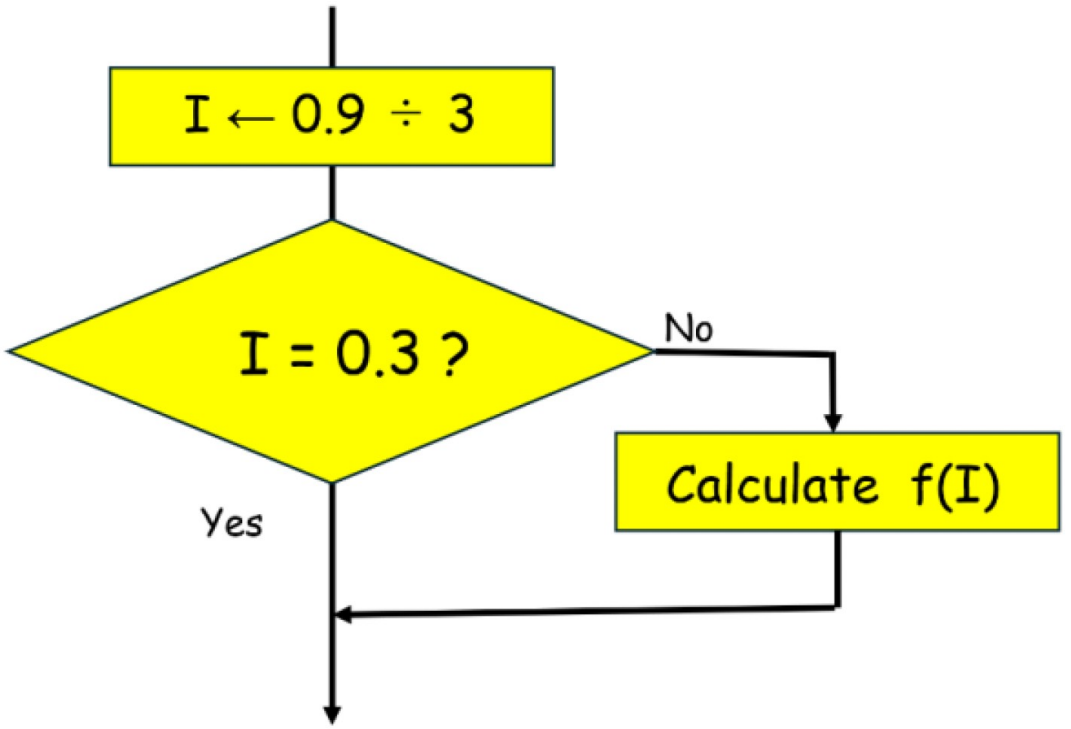


# Double-precision IEEE 754 Floating Point standard:



## Avoid relying on exact equality for floating-point calculations in most programming languages.

The result of the first box cannot guarantee to be the value 0.3.



# The Character Data Type

## Character Data Type

- To store text, you use the character data type. To store one character in a variable, you use the char keyword and place the character in single quotation marks.
- Example:

```
char cLetter = 'A';
```

## Escape Sequence

- The combination of a backlash (\) and a special character is called an escape sequence.
- Example:

| | |
|---|---|
| `\n` | move to the next line |
| `\t` | move to the next tab |

# Arithmetic Operators

- Arithmetic operators are used to perform mathematical calculations, such as addition, subtraction, multiplication, and division.

| Operator | Description |
|---|---|
| + | Add two operands |
| - | Subtracts one operand from another operand |
| * | Multiplies one operand by another operand |
| / | Divides one operand by another operand |
| % | Divides two operands and returns the remainder |

- A simple arithmetic expression consists of an arithmetic operator connecting two operands in the form:

<operand1> **<operator>** <operand2>

# Arithmetic Operators – Example 2.2.1

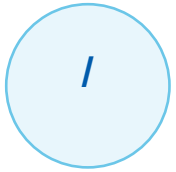
```
#include <iostream>
using namespace std;
int main()
{
    cout << "15.0 plus 2.0 equals "
         << (15.0 + 2.0) << '\n'
         << "15.0 minus 2.0 equals "
         << (15.0 - 2.0) << '\n'
        << "15.0 times 2.0 equals "
         << (15.0 * 2.0) << '\n'
        << "15.0 divided by 2.0
equals"
         << (15.0 / 2.0) << '\n';
    return 0;
}
```

**Output**

```
15.0 plus 2.0 equals 17
15.0 minus 2.0 equals 13
15.0 times 2.0 equals 30
15.0 divided by 2.0 equals 7.5
```

# Integer Division and % operator

## Integer Division




- The division of two integers yields an integer result.
- Thus the value of 15/2 is 7.

## Modulus Division




- The modulus % operator produces the remainder of an integer division.
- Example:
  `9 % 4` is `1`
  `17 % 3` is `2`
  `14 % 2` is `0`

# Operator Precedence and Associativity

- Expressions containing multiple operators are evaluated by the priority, or precedence, of the operators.

| Operator | Associativity |
|---|---|
| unary - | Right to left |
| * / % | Left to right |
| + - | Left to right |

- Example 1: 8 + 5 * 7 % 2 * 4




- Example 2: 9 / 2 * 3 - 10 * 4 % 3

03

# VARIABLES AND DECLARATIONS

**Learn how to define identifiers, declare variables, and allocate memory to store program data.**

## One of the most important aspects of programming is storing and manipulating the values stored in variables.

Variable names or identifiers in C++ can made up of any combination of letters, digits, or underscores selected according to the following rules:

01 **Identifiers must begin within an uppercase or lowercase ASCII letter or an underscore (_).**

02 **You can use digits in an identifier, but not as the first character. You are not allowed to use special characters such as $, &, *, or %.**

03 **Reserved words cannot be used for variable names.**

# Identifiers

## Some valid identifiers



- `my_variable` (Python style)
- `MyVariable` (Pascal style)
- `myVariable` (camel style 🐫, suggested style for C++)
- `Temperature, x1, x2`

## Some invalid identifiers



- `%x1`
- `_my%var`
- `X$2`

# Declaration Statements

## Syntax

- In C++ you can declare the data types of variables using the syntax: `<type> <name>;`
- The `type` portion refers to the data type of the variable. The data type determines the type of information that can be stored in the variable.

## Example

- `int count;`
- `float sum;`

# Rules of variable declaration

- Example:

```
int num = 15;
float   grade1   =
87.0;
```

01 A variable must be declared before it can be used.

02 Declaration statements can also be used to store an initial value into declared variables.

03 The declaration statement gives information to the compiler rather than a step in the algorithm.

# Example 2.3.1

```
#include <iostream>
using namespace std;
int main()
{
    float grade1 = 85.5;
    float grade2 = 97.0;
    float total, average;
    total = grade1 + grade2;
    // divide the total by 2.0
    average = total/2.0;
    cout << "The average grade is "
         << average << endl;
    return 0;
}
```

## Output & Explanation

- The output of the above program:
  `The average grade is 91.25`
- Let's notice the two assignment statements in the above program:
  `total = grade1 + grade2;`
  `average = total/2.0;`
- Each of these statements is called an assignment statement because it tells the computer to assign (store) a value into a variable.

04

# ASSIGNMENT STATEMENTS

**Master assignment operations, compound assignment variations, and the behavior of increment/decrement operators (++, --).**

## How do we place data items into variables?

- Assignment statement examples:
  - `length = 25;`
  - `cMyCar = "Mercedes";`
  - `sum = 3 + 7;`
  - `newtotal = 18.3 * amount;`

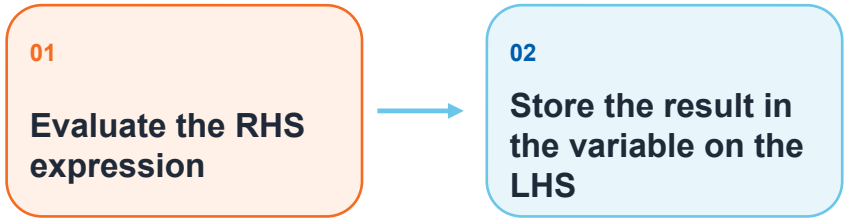
01 **Method 1: Read in values typed at the keyboard by the user**

02 **Method 2: Use an assignment statement**

03 **Assignment operator (=) are used for assigning a value to a variable and for performing computations.**

# Assignment statement (cond.)

- The assignment statement has the syntax: `<variable> = <expression>;`
  - Expression is any combination of constants, variables, and function calls that can be evaluated to yield a result.
- The order of events when the computer executes an assignment statement is:




**Note:**
- **The equal sign (=) here does not have the same meaning as an equal sign in mathematics.**
- **Each time a new value is stored in a variable, the old one is overwritten.**

# Example 2.4.1

**This program calculates the volume of a cylinder, given its radius and height.**

```
#include <iostream>
using namespace std;
int main()
{
    float radius = 2.5, height = 16.0, volume;
    volume = 3.1416 * radius * radius * height;
    cout << "The volume of the cylinder is " << volume << endl;
    return 0;
}
```

**Output**

- `The volume of the cylinder is 314.16`

# Assignment Variations

- C++ includes other arithmetic operators in addition to the equal sign of assignment.

| Operator | Example | Meaning |
|---|---|---|
| = | iNum1 = iNum2 | |
| += | iNum1 += iNum2 | iNum1 = iNum1 + iNum2 |
| -= | iNum1 -= iNum2 | iNum1 = iNum1 - iNum2 |
| *= | iNum1 *= iNum2 | iNum1 = iNum1 * iNum2 |
| /= | iNum1 /= iNum2 | iNum1 = iNum1 / iNum2 |
| %= | iNum1 %= iNum2 | iNum1 = iNum1 % iNum2 |

- So sum += 10 is equivalent to sum = sum + 10

# Data Type Conversion across Assignment Operator

## Data type conversion

- Data type conversion can take place across assignment operators, i.e., the value of the expression on the right side is converted to the data type of the variable on the left side.

## Example

- If temp is an integer variable, the assignment temp = 25.89 causes the integer value 25 to be stored in the integer variable temp.

# Increment and decrement operators

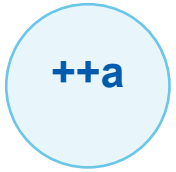
- For the special case in which a variable is either increased or decreased by 1, C++ provides two unary operators: **increment operator and decrement operator.**

| Operator | Description |
|---|---|
| ++ | Increase an operand by a value of one |
| -- | Decrease an operand by a value of one |

- The **increment (++)** and **decrement (--)** unary operators can be used as prefix or postfix operators to increase or decrease value.

# Increment and decrement operators (cond.)

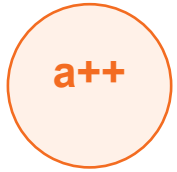
## Prefix operator




**A prefix operator is placed before a variable and returns the value of the operand after the operation is performed.**

- `b = ++a;  // prefix way`
- Will first increase the value of a (equal to 5) to 6, and then assign that new value to b.
- It is equivalent to `a = a + 1; b = a;`

## Postfix operator




**A postfix operator is placed after a variable and returns the value of the operand before the operation is performed.**

- `b = a++;  // postfix way`
- Will first assign the value of a (equal to 5) to b, and then increase the value of a to 6.
- It is equivalent to `b = a; a = a + 1;`

**The decrement operators (--a, a–) are used in a similar way.**

# Example 2.4.2

```
#include <iostream>
using namespace std;
int main()
{
   int c;
   c = 5;
   cout << c << endl;          // print
5
   cout << c++ << endl;        // print
5 then post-increment
   cout << c << endl << endl; // print
6
   c = 5;
   cout << c << endl;          // print
5
   cout << ++c << endl;        // pre-
increment then print 6
   cout << c << endl;          // print
6
   return 0;
}
```

**Output**

```
5
5
6

5
6
6
```

05

# INPUT WITH THE CIN OBJECT

**Master assignment operations, compound assignment variations, and the behavior of increment/decrement operators (++, --).**

# Program Input Using The cin Object

## So far, our programs have been limited since all their data must be defined within the program source code.

We now learn how to write programs that enable data to be entered via the keyboard while the program is running using the Standard Input Stream.

01 **The `cin` object reads in information from the keyboard via the standard input stream.**

02 **The extraction operator `(>>)` retrieves information from the input stream.**

03 **When the statement `cin >> num1;` is encountered, the computer stops program execution and accepts data from the keyboard. When a data item is typed, the `cin` object stores the item into the variable listed after the `>>` operator.**

# Example 2.5.1

```
#include <iostream>
using namespace std;
int main()
{
	float num1, num2, product;
	 cout << "Please type in a
number: ";
	cin >> num1;
	  cout << "Please type in
another number: ";
	cin >> num2;
	product = num1 * num2;
	 cout << num1 << " times " <<
num2 << " is " << product << endl;
	return 0;
}
```

**Output**

```
Please type in a number: 30
Please type in another number:
0.05
30 times 0.05 is 1.5
```

# Example 2.5.2

```
#include <iostream>
using namespace std;
int main()
{
	int num1, num2, num3;
	float average;

	 cout << "Enter three integer
numbers: ";
	cin  >> num1 >> num2 >> num3;
	 average = (num1 + num2 +
num3) / 3.0;
	 cout << "The average of the
numbers is " << average << endl;
	return 0;
}
```

**Output**

```
Enter three integer numbers: 22 56 73
The average of the numbers: 50.333333
```

06

# MATHEMATICAL LIBRARY FUNCTIONS

**Extend numerical capabilities by leveraging standard mathematical functions provided in the <cmath> library.**

# Using Mathematical Library Functions

- C++ provides standard library functions that can be included in a program.
- If your program uses the mathematical function *sqrt(),* it should have the preprocessor command #include <cmath> in the beginning of the program.
  - #include <cmath>: C++ version of the math header. The functions are placed in the std namespace. This is the **recommended way** in modern C++.
  - #include <math.h>: C version of the math header, inherited from the C standard library.

| Function Name | Description | Return Value |
|---|---|---|
| abs(a) | absolute value | Same data type as argument |
| log(a) | natural logarithm | double |
| sin(a) | sine of a (a in radians) | double |
| cos(a) | cosine of a (a in radians) | double |

# Using Mathematical Library Functions (cond.)

| Function Name | Description | Return Value |
|---|---|---|
| tan(a) | tangent of a (a in radians) | double |
| log10(a) | common log (base 10) of a | double |
| pow(a1, a2) | a1 raised to the a2 power | double |
| exp(a) | $e^a$, e~2.71828 | double |
| sqrt(a) | square root of a | double |

- Except for abs(a), all take a double argument and return a value of type double.
- **Note:** The typical floating-point types and their sizes in C++ are:
  - float: **4** bytes
  - double: **8** bytes

07

# FORMATTING THE OUTPUT

**Format console output cleanly and accurately using stream manipulators like setw and setprecision.**

# Formatting for program output

**Besides displaying correct results, a program should present its results attractively with good formats.**

Stream manipulators are used to format the program output

01 **Stream manipulator functions are special stream functions that change certain characteristics of the input and output.**

02 **The main advantage of using manipulator functions is that they facilitate the formatting of the input and output streams.**

# Stream Manipulators

## Width manipulator

setw()

**This manipulator is used to specify the minimum number of the character positions on the output field a variable will consume.**

## Precision manipulator

setprecision()

**This manipulator is used to control the number of digits of an output stream display of a floating-point value.**

- Setprecision(2) means **2** digits of precision to the right of the decimal point (this is only true if fixed mode is enabled).

# Example 2.7.1

```
cout << "|" << setw(10)
      << setprecision(3) << fixed <<
25.67 << "|";
```

## Output & Explanation

- Cause the printout: `|     25.670|`
- Without the "fixed" keyword, the precision starts at the beginning of the number. With the "fixed" keyword, it starts at the decimal point.
- To reset the behavior after using "fixed", we can use "defaultfloat"

# Example 2.7.2

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
  cout << setw(3) << 6 << endl
       << setw(3) << 18 << endl
       << setw(3) << 124 << endl
       << "---\n"
       << (6+18+124) << endl;
  return 0;
}
```

**Output**

```
  6
 18
124
---
148
```

# Key Takeaways

- **Foundations:** C++ program structure with main(), preprocessor directives (#include), namespaces (std), and fundamental data types (int, float, char).
- **Mechanisms:** IEEE 754 floating-point approximation, operator precedence/associativity, variable lifecycle, type conversion, and prefix vs. postfix (++ / --).
- **Applications:** Interactive console I/O using cin/cout, mathematical computation with <cmath>, and clean data presentation via <iomanip> (setw, setprecision, fixed).
- **Next steps:** Preparing for chapter 3 (Selection Structures)

**Thank you for listening!**